Java & Spring Boot Interview Questions with Answers

(Simple, easy-to-understand explanations)

---

1. Core Java — Strings, Objects, Collections

1. Why is `String` immutable in Java? Once a `String` is created, it can never change. If you "modify" it, Java actually creates a new String object. This is done for security (passwords, URLs), thread-safety (many threads can read it safely), and performance (Java reuses Strings from a special memory area called the String Pool instead of creating duplicates).

2. Why must `equals()` and `hashCode()` follow a contract? The rule is: if two objects are equal (`equals()` returns true), they must have the same `hashCode()`. Collections like `HashMap` and `HashSet` use the hash code first to find the "bucket" where an object should be, then use `equals()` to confirm. If you break this rule, the object can get lost — you store it, but the collection can never find it again.

3. How does `HashMap` work internally? A `HashMap` stores data in key-value pairs. When you add a key, Java calculates its `hashCode()`, uses that to decide which "bucket" (slot in an array) to place it in. If two keys land in the same bucket, they are stored together as a small list (or a tree, if the list gets too long). When you look up a key, Java again calculates the hash code to jump straight to the right bucket instead of searching everything.

4. What happens when two keys have the same hashcode? This is called a "collision." Both key-value pairs go into the same bucket. Inside the bucket, Java uses `equals()` to tell the keys apart. In modern Java, if too many keys collide in one bucket, that bucket turns into a balanced tree instead of a plain list, so lookups stay fast.

5. `HashMap` vs `HashSet` `HashMap` stores key-value pairs (e.g., name → age). `HashSet` stores only unique values, no keys. In fact, internally, `HashSet` is built using a `HashMap` where your values become the keys and a dummy constant is used as the value.

6. Why can `HashSet` detect duplicate objects incorrectly if `equals()` and `hashCode()` are implemented badly? `HashSet` relies on `hashCode()` and `equals()` to decide if two objects are "the same." If you write these methods incorrectly (or don't override them at all), two objects that should be treated as duplicates might get different hash codes and both get added, or two different objects might wrongly be treated as the same and one gets rejected.

7. `ArrayList` vs `LinkedList` — when would you use each? `ArrayList` stores data in a resizable array — great for fast random access (`get(index)`) but slower to insert/delete in the middle since elements need to shift. `LinkedList` stores data as a chain of nodes — inserting/removing is fast if you already have a reference to the position, but accessing a random index is slow because it has to walk through the chain. In practice, `ArrayList` is used far more often because most real-world tasks need fast reading.

8. What happens if you modify a collection while iterating over it? You'll usually get a `ConcurrentModificationException`. Java's iterators keep an internal counter to track changes; if the collection is modified (add/remove) outside the iterator while looping, Java detects the mismatch and throws an error. To safely remove items while looping, use the iterator's own `remove()` method or a `CopyOnWriteArrayList`/`ConcurrentHashMap` for concurrent situations.

9. When would you use `Optional`, and when should you avoid it? Use `Optional` as a return type to clearly tell the caller "this method might not return a value" — it forces them to handle the missing case instead of accidentally hitting a `NullPointerException`. Avoid using `Optional` for class fields, method parameters, or in performance-critical loops, since it adds extra object overhead and isn't meant for those purposes.

10. Streams vs traditional loops — when would you prefer one over the other? Streams (`list.stream().filter(...).map(...)`) are great for clean, readable code when transforming or filtering data, and they support easy parallel processing. Traditional loops are better when you need complex logic, early exits (`break`), or when performance is extremely critical, since loops have less overhead.

11. Method Overloading Having multiple methods with the same name but different parameters (different number or type of arguments) in the same class. Java decides which one to call based on the arguments you pass, at compile time.

12. Interface vs Abstract Class An interface defines *what* a class should do (a contract) — traditionally only method signatures, though modern Java allows default and static methods too. A class can implement many interfaces. An abstract class can have both complete and incomplete methods, and can hold state (fields/constructors) — but a class can extend only one abstract class.

13. Class vs Interface A class is a blueprint used to create real objects with actual data and full method implementations. An interface is a contract that only says what methods must exist, without necessarily implementing them, and a class "promises" to fulfill that contract by implementing it.

14. Exception Handling and different ways to handle exceptions Exceptions are unexpected problems during program execution. Java gives you `try-catch` blocks to catch and handle errors, `finally` to run cleanup code no matter what, `throw` to raise a custom exception, and `throws` in a method signature to declare that a method might throw an exception. You can also create your own custom exception classes for specific business errors.

---

2. Multithreading & Concurrency

15. `synchronized` vs `volatile` — when would you use each? `synchronized` locks a block of code or method so only one thread can run it at a time — use it when multiple threads read AND write shared data. `volatile` only guarantees that all threads see the latest value of a variable (no caching issues) — use it when one thread writes and others just read, without needing locking.

16. How would you identify a deadlock in a Java application? A deadlock happens when two or more threads are stuck waiting for each other's locks forever. You can spot it by taking a "thread dump" (a snapshot of all threads) using tools like `jstack`, VisualVM, or JDK Mission Control — Java even directly labels threads stuck in a deadlock in the dump.

17. `Runnable` vs `Callable` — when would you choose each? `Runnable` represents a task that doesn't return a result and can't throw checked exceptions. `Callable` represents a task that *does* return a result (via `Future`) and can throw checked exceptions. Use `Callable` whenever you need the outcome of a background task.

18. Why would you use `ExecutorService` instead of creating threads manually? Creating raw threads manually is wasteful and hard to manage — you'd have to handle thread creation, reuse, and shutdown yourself. `ExecutorService` manages a pool of reusable threads for you, so you just submit tasks and it handles scheduling, reuse, and resource limits efficiently.

19. What problem does `CompletableFuture` solve? It lets you write asynchronous (non-blocking) code in a clean, readable way. Instead of blocking and waiting for a background task to finish, you can chain what should happen next (`thenApply`, `thenCombine`, etc.), combine multiple async tasks, and handle errors — all without freezing the main thread.

20. What happens if an exception is thrown inside a thread? If it's not caught inside the thread's own code, the thread simply terminates and the exception is handled by the thread's "uncaught exception handler" (by default, it just prints the stack trace). It does NOT crash the whole application or affect other threads — only that one thread dies.

21. `HashMap` vs `ConcurrentHashMap` — what's the real difference? `HashMap` is not thread-safe — if multiple threads modify it at once, you can get data corruption or infinite loops. `ConcurrentHashMap` is designed for multi-threaded use: it divides the map into segments/buckets and only locks the small part being changed, allowing multiple threads to read and write safely and efficiently at the same time.

---

3. JVM, Memory & Garbage Collection

22. Heap vs Stack — what is stored where? The **Stack** stores method calls and local variables — it's fast and each thread has its own stack, and memory is automatically freed when a method finishes. The **Heap** stores actual objects (created with `new`) and is shared across all threads — it's managed by the Garbage Collector.

23. How does the JVM memory model work? JVM memory is divided into areas: the Heap (for objects), Stack (for method calls/local variables, one per thread), Metaspace (for class metadata), and PC registers/native stacks. The JVM also defines rules about how threads see changes to shared variables — this is the "memory model" that keeps multithreaded code consistent.

24. Explain the Class Loading lifecycle. Before a class can be used, it goes through: **Loading** (reading the .class file into memory), **Linking** (which has three sub-steps: Verification — checking the bytecode is valid; Preparation — allocating memory for static variables; Resolution — replacing symbolic references with real ones), and finally **Initialization** (running static initializers and assigning static variable values).

25. What are Bootstrap, Platform, and Application ClassLoaders? These load classes in a hierarchy. **Bootstrap ClassLoader** loads core Java classes (like `java.lang.*`). **Platform ClassLoader** loads Java platform/extension classes. **Application ClassLoader** loads your own application's classes (from the classpath). Each loader asks its parent to load a class first before trying itself (called delegation).

26. How does Garbage Collection work internally? GC automatically frees memory used by objects that are no longer reachable (no live reference points to them). Most GCs use a "generational" approach: new objects go into the Young Generation; objects that survive multiple GC cycles move to the Old Generation. GC repeatedly scans for unreachable objects and reclaims their memory.

27. G1 GC vs ZGC vs Serial GC? Serial GC uses a single thread and pauses the whole application — fine for small apps. **G1 GC** (Garbage First) divides the heap into regions and is designed to balance throughput and predictable pause times — it's the default for most modern apps. **ZGC** is built for very large heaps with extremely low pause times (sub-millisecond), ideal for latency-sensitive, huge-memory applications.

28. What triggers a Full GC, and why can it affect API latency? A Full GC happens when the Old Generation fills up, or memory pressure forces cleaning of the whole heap (not just young objects). Full GC typically causes a "stop-the-world" pause where all application threads freeze, which can make API requests suddenly take much longer or time out.

29. How do you identify memory leaks in production? Watch heap usage trends over time (it should go up and down with GC, not just keep climbing). Take heap dumps and analyze them with tools like Eclipse MAT or VisualVM to see which objects are piling up and who's still referencing them.

30. How can a Java application have a memory leak despite having Garbage Collection? GC only removes objects with no live references. If your code accidentally keeps a reference to an object it no longer needs (e.g., objects added to a static list/map and never removed, unclosed resources, or listeners that are never unregistered), GC can't touch them — and memory keeps growing even though GC is running fine.

31. What is a Heap Dump and when would you analyze it? A heap dump is a snapshot of all objects in memory at a point in time. You analyze it when you suspect a memory leak or `OutOfMemoryError`, to see exactly what objects exist, how many, and what's holding references to them.

32. How does the JIT Compiler improve performance? The JVM initially runs bytecode through an interpreter (slower). The Just-In-Time (JIT) compiler watches which code runs frequently ("hot" code) and compiles it directly into fast native machine code at runtime, so repeated code runs much faster over time.

33. What is Escape Analysis? This is a JIT optimization technique where the JVM checks if an object "escapes" the method it was created in (i.e., is it used elsewhere or just locally?). If an object never escapes, the JVM can allocate it on the stack instead of the heap (or skip allocation entirely), which is much faster and reduces GC work.

34. What is Metaspace and how is it different from PermGen? Both store class metadata (like class structure, method info). Older Java versions used PermGen, which had a fixed size and often caused `OutOfMemoryError: PermGen space`. Since Java 8, PermGen was replaced with Metaspace, which lives in native (non-heap) memory and grows dynamically, mostly avoiding that fixed-size problem.

35. How do you troubleshoot `OutOfMemoryError`? Check the error message type (Heap space, Metaspace, GC overhead limit, etc.) since each points to a different cause. Take a heap dump at the time of the

crash (`-XX:+HeapDumpOnOutOfMemoryError`), analyze it to find what's consuming memory, and check for memory leaks, undersized heap settings, or genuinely high memory needs.

36. How do you troubleshoot high CPU usage in JVM applications? Find the process ID, then find which threads inside it are using high CPU (using `top -H` on Linux). Take a thread dump at that moment and match the high-CPU thread IDs to see exactly what code is running — often it's an infinite loop, excessive GC, or heavy computation.

37. What tools do you use for JVM performance analysis? Common tools: `jstack` (thread dumps), `jmap`/heap dump tools, VisualVM, JDK Mission Control (JFR - Java Flight Recorder), Eclipse MAT (memory analysis), and APM tools like New Relic or Dynatrace for production monitoring.

38. Your Java API suddenly becomes slow in production. What JVM-level metrics would you check first? Check: GC frequency and pause times (frequent Full GCs are a red flag), heap usage over time, CPU usage, thread count and thread states (are threads blocked/waiting?), and whether there's a spike in traffic or a slow downstream dependency (database, external API) causing threads to pile up.

---

4. Java Design Patterns

39. What problem does the Factory Pattern solve? It hides the logic of *how* an object is created from the code that *uses* the object. Instead of calling `new` directly and hardcoding a specific class, you ask a "factory" method for an object, and it decides which exact class to instantiate — making the code more flexible to change.

40. Factory Pattern vs Abstract Factory Pattern? The Factory Pattern creates one type of object based on some input. The Abstract Factory Pattern is a "factory of factories" — it creates families of related objects (e.g., a UI factory that creates matching buttons, checkboxes, and menus for either Windows or Mac style) without specifying their exact classes.

41. When should you use the Builder Pattern? Use it when an object has many optional fields/parameters and you want to avoid a constructor with a huge number of arguments. The Builder lets you set only the fields you need, one by one, in a readable, chainable way (e.g., `Person.builder().name("John").age(30).build()`).

42. Why is Builder preferred for immutable objects? Immutable objects can't be changed after creation, so all their values must be set at once, at construction time. If there are many fields, a normal constructor becomes unreadable. Builder lets you gather all the values step by step first, then create the final unchangeable object in one clean `build()` call.

43. How do you implement a thread-safe Singleton? A Singleton ensures only one instance of a class ever exists. To make creation thread-safe, common approaches are: using an `enum` (simplest and safest), using a static inner class (lazy loading, thread-safe by JVM class-loading guarantees), or "double-checked locking" with `synchronized` and `volatile`.

44. What are the drawbacks of Singleton? It can make unit testing harder (since it's a global, shared state that's hard to mock or reset), can hide dependencies (code secretly depends on the singleton

instead of declaring it clearly), and can cause issues in multi-threaded or distributed systems if not designed carefully.

45. Explain the Strategy Pattern with a real-world example. Strategy lets you swap out an algorithm/behavior at runtime without changing the code that uses it. Example: a payment system where you can pay by `CreditCardStrategy`, `PayPalStrategy`, or `UpiStrategy` — the checkout code just calls `strategy.pay()` without caring which one is actually plugged in.

46. When would you use the Observer Pattern? Use it when one object (the "subject") needs to automatically notify many other objects ("observers") whenever its state changes, without those objects constantly checking/polling. Example: a news publisher notifying all subscribers whenever a new article is posted.

47. How is Observer used in event-driven systems? Event-driven systems are built entirely on this idea: something happens (an event, like a button click or new order), and all "listeners" registered for that event get automatically notified/triggered — exactly the observer pattern, just usually with a name like "listener" or "event handler."

48. Adapter Pattern vs Decorator Pattern? Adapter is used to make two incompatible interfaces work together — like a plug adapter for different socket types, it *translates* one interface into another. Decorator is used to *add new behavior* to an object dynamically without changing its class — like adding toppings to a base pizza, wrapping the original object with extra functionality.

49. What problem does the Proxy Pattern solve? Proxy provides a stand-in/placeholder object that controls access to the real object. It's used for things like lazy loading (create the real object only when actually needed), access control (check permissions before allowing access), logging, or caching — all without the caller knowing they're talking to a proxy instead of the real thing.

50. How is the Template Method Pattern different from Strategy? Template Method defines the *skeleton* of an algorithm in a base class, letting subclasses override specific steps — the overall structure stays fixed. Strategy lets you swap the *entire* algorithm/behavior at runtime by plugging in a different strategy object — there's no fixed skeleton being reused.

51. What is Dependency Injection and which pattern does it use? Dependency Injection means an object doesn't create its own dependencies (other objects it needs) — instead, they are "injected" (given to it) from outside, usually by a framework like Spring. This follows the broader "Inversion of Control" (IoC) principle, and is itself considered a design pattern for decoupling code.

52. Which design patterns are commonly used in Spring Framework? Singleton (Spring beans are singleton by default), Factory Pattern (`BeanFactory`, `ApplicationContext` create beans for you), Proxy Pattern (used heavily for AOP and `@Transactional`), Template Method (e.g., `JdbcTemplate`), and Observer Pattern (Spring's event publishing system).

53. Which design patterns are most commonly used in Microservices? Common ones include: Circuit Breaker (stop calling a failing service repeatedly), API Gateway (single entry point for all services), Saga Pattern (manage transactions across multiple services), Strategy Pattern (swap implementations per service), and Observer/Event-Driven patterns (services communicating via events/messages).

---

5. Spring Boot, REST APIs & Database

54. REST APIs — overview REST (Representational State Transfer) is a style for building web APIs where you interact with "resources" (like `/users`, `/orders`) using standard HTTP methods: `GET` (read), `POST` (create), `PUT/PATCH` (update), and `DELETE` (remove). Data is usually exchanged in JSON format.

55. Steps to create REST APIs using Spring Boot 1) Create a Spring Boot project (via Spring Initializr). 2) Create a Model/Entity class representing your data. 3) Create a Repository interface (extends `JpaRepository`) for database access. 4) Create a Service class with business logic. 5) Create a Controller class with `@RestController` and map endpoints using `@GetMapping`, `@PostMapping`, etc. 6) Test the API using Postman or similar tools.

56. How do you create and integrate REST APIs? You define a Controller class, annotate it with `@RestController`, and write methods annotated with HTTP mappings (`@GetMapping("/users")`, etc.). Inside those methods, you call your Service layer to fetch/process data, and return the result — Spring Boot automatically converts Java objects to JSON for the response.

57. How do you test REST APIs? Manually, using tools like Postman or cURL to send requests and check responses. Automatically, using testing frameworks — `MockMvc` or `@SpringBootTest` with `TestRestTemplate` to write integration tests that verify status codes, response bodies, and behavior.

58. Explain the flow from REST API → Controller → Service → Repository → Database. A client sends an HTTP request → it hits the **Controller**, which handles the request/response mapping → the Controller calls the **Service** layer, which contains the actual business logic → the Service calls the **Repository**, which talks to the **Database** to fetch or save data → the result travels back up through Service → Controller → and finally back to the client as a response.

59. What happens when an API request comes to your Spring Boot application? The embedded server (like Tomcat) receives the HTTP request. Spring's `DispatcherServlet` (front controller) intercepts it, figures out which Controller method should handle it based on the URL/HTTP method, passes through any filters/interceptors, executes the method (which usually calls Service → Repository), and then sends the result back as an HTTP response.

60. How do you handle exceptions in Spring Boot? You can use `@ExceptionHandler` inside a controller for specific handling, or better, create a global exception handler class annotated with `@ControllerAdvice`/`@RestControllerAdvice` that catches exceptions across the whole application and returns clean, consistent error responses (with proper status codes and messages).

61. JDBC implementation JDBC (Java Database Connectivity) is the low-level API for connecting Java to a database. Basic steps: load the driver, open a `Connection`, create a `Statement`/`PreparedStatement`, execute the SQL query, process the `ResultSet`, then close the connection. Frameworks like Spring Data JPA are built on top of JDBC to simplify this.

62. Spring Boot application flow and database connectivity Spring Boot reads database configuration from `application.properties`/`.yml` (URL, username, password, driver). It uses this to auto-configure a `DataSource` bean at startup. Your Repository interfaces (using Spring Data JPA) then use this DataSource under the hood to run queries without you writing raw SQL boilerplate.

63. How did you integrate PostgreSQL with Spring Boot? Add the PostgreSQL driver dependency in `pom.xml`/`build.gradle`, configure the database URL, username, and password in `application.properties`, and Spring Boot auto-configures the connection. Then you define Entity classes and Repository interfaces to interact with the tables.

64. How does Spring Data JPA work with PostgreSQL? Spring Data JPA sits on top of JPA/Hibernate. You define an Entity class mapped to a table (using `@Entity`, `@Table`, `@Column`), and a Repository interface extending `JpaRepository`. Spring automatically generates the SQL for common operations (save, find, delete) — you rarely need to write SQL yourself for basic operations.

65. How do you check whether data is correctly inserted/updated in PostgreSQL? You can query the table directly using a SQL client (like pgAdmin or `psql`) with a `SELECT` statement, check application logs (if SQL logging is enabled), or call your own API's `GET` endpoint to fetch and verify the data through the application itself.

---

6. Transactions (`@Transactional`)

66. What is a transaction? A transaction is a group of database operations that are treated as a single unit — either **all** of them succeed together, or **none** of them do. This keeps data consistent, e.g., transferring money: debiting one account and crediting another must both happen, or neither should.

67. What is rollback? Rollback means undoing all the changes made during a transaction if something goes wrong partway through, so the database returns to its state before the transaction started — preventing partial/incorrect updates.

68. How do you handle transactions in Spring Boot? Simply annotate a Service method with `@Transactional`. Spring wraps the method in a database transaction automatically — if the method completes normally, it commits; if a runtime exception is thrown, it rolls back.

69. What is `@Transactional`? It's a Spring annotation you put on a method or class to say "run this as a single database transaction." Spring handles beginning the transaction, committing it on success, and rolling it back on failure — without you writing manual transaction code.

70. Give a real-time example where rollback is required. An e-commerce order: you first reduce stock quantity, then create the order record, then charge the payment. If the payment fails at the last step, you need the stock reduction and order creation to also be undone (rolled back) — otherwise, stock is wrongly reduced for an order that never actually succeeded.

71. How does `@Transactional` work internally? Spring creates a **proxy** object around your bean. When you call a `@Transactional` method, you're actually calling the proxy first — it starts a transaction, then calls your real method, and based on whether it completes normally or throws an exception, the proxy commits or rolls back the transaction.

72. Why does `@Transactional` sometimes not work? Common reasons: calling the `@Transactional` method from within the *same class* (self-invocation bypasses the proxy), the method isn't `public`, the

exception thrown is a checked exception (by default only unchecked/runtime exceptions trigger rollback), or the class isn't managed as a Spring bean at all.

73. What exceptions trigger transaction rollback? By default, Spring rolls back only on unchecked exceptions (`RuntimeException` and its subclasses) and `Error`. Checked exceptions do NOT trigger rollback by default — unless you explicitly configure it using `@Transactional(rollbackFor = Exception.class)`.

74. What is the difference between `REQUIRED` and `REQUIRES_NEW`? `REQUIRED` (the default) means: if a transaction already exists, join it; if not, create a new one. `REQUIRES_NEW` always suspends any existing transaction and starts a completely new, independent one — so if the new transaction rolls back, it doesn't affect the outer one.

75. Explain all transaction propagation types. - **REQUIRED**: Join existing transaction, or create a new one. - **REQUIRES_NEW**: Always start a new, independent transaction (suspends any existing one). - **SUPPORTS**: Join a transaction if one exists, otherwise run without one. - **NOT_SUPPORTED**: Always run without a transaction (suspends any existing one). - **MANDATORY**: Must run within an existing transaction, or throw an error. - **NEVER**: Must NOT run within a transaction, or throw an error. - **NESTED**: Runs within a nested transaction (using savepoints) inside an existing one, if present.

76. What is transaction isolation? It controls how much one transaction can "see" of another transaction's uncommitted changes when they run at the same time. Higher isolation means more consistency but less concurrency (slower); lower isolation means more speed but higher risk of seeing inconsistent data.

77. Explain all isolation levels. - **READ UNCOMMITTED**: Can see other transactions' uncommitted (dirty) changes. - **READ COMMITTED**: Only sees data that's been committed by other transactions (most common default). - **REPEATABLE READ**: Guarantees the same data if you read a row twice in the same transaction. - **SERIALIZABLE**: Highest level — transactions run as if fully one-at-a-time, preventing all anomalies but slowest.

78. What causes dirty reads? A dirty read happens when one transaction reads data that another transaction has changed but not yet committed. If that second transaction then rolls back, the first transaction ends up having used data that never truly existed.

79. What causes phantom reads? A phantom read happens when a transaction runs the same query twice, and gets a *different number of rows* the second time — because another transaction inserted or deleted rows matching that query in between.

80. Why should API calls be avoided inside transactions? External API calls can be slow or fail unpredictably. If you make one inside a database transaction, the transaction (and its database locks) stays open the whole time the API call is happening — this can block other operations, slow the whole system down, and cause timeouts.

81. What happens during nested transactions? With `NESTED` propagation, a "savepoint" is created before running the nested part. If the nested part fails, only it rolls back to that savepoint — the outer transaction can still continue and commit its own other changes (this depends on database/driver support for savepoints).

82. What is transaction synchronization? It's a mechanism where extra actions (like clearing a cache, sending a notification, or closing a resource) are registered to run automatically at specific points of a transaction's lifecycle — such as right after it commits or rolls back.

83. How do you debug transaction issues? Enable SQL/transaction logging (e.g., set logging level for Spring's transaction manager and Hibernate to DEBUG) to see exactly when transactions begin, commit, or rollback, check for self-invocation or checked-exception issues, and verify the `@Transactional` annotation is correctly placed on a Spring-managed, public method.

84. What is the self-invocation problem? Since `@Transactional` relies on a proxy wrapping your class, if a method inside a class calls *another* `@Transactional` method on `this` (itself) directly, it bypasses the proxy — so the transactional behavior is silently ignored.

85. How does Spring use proxies for transaction management? When a bean has `@Transactional` methods, Spring creates a proxy object (using either JDK dynamic proxies for interfaces, or CGLIB for classes) that wraps your actual bean. All calls go through this proxy first, letting Spring insert transaction start/commit/rollback logic before and after your real method runs.

---

7. Docker & Deployment

86. How do you use Docker in your project? Docker packages the application along with everything it needs (Java runtime, dependencies, configuration) into a single "container image," so it runs the same way on any machine. Typically, you write a `Dockerfile` to build the image, and use Docker Compose to run the app together with its dependencies like a database.

87. What is a Dockerfile? It's a text file with step-by-step instructions on how to build a Docker image — e.g., which base image to start from (like a JDK image), which files to copy in, how to build the app, and what command to run when the container starts.

88. What is Docker Compose? A tool that lets you define and run multiple containers together using one YAML configuration file (`docker-compose.yml`) — for example, running your Spring Boot app and a PostgreSQL database as separate containers that can talk to each other, all started with a single command.

89. Why do we use Docker Compose? It makes managing multi-container applications much easier — instead of manually starting each container with long commands, you define everything (services, networks, environment variables, ports) once in a file, and start/stop the whole setup with one simple command.

90. How do you run Spring Boot and PostgreSQL together using Docker Compose? In `docker-compose.yml`, you define two services: one for your Spring Boot app (built from your Dockerfile) and one for PostgreSQL (using the official postgres image with environment variables for username/password/db name). You connect them via a shared Docker network and configure your app's database URL to point to the PostgreSQL service's name.

91. How do you deploy your application? Common approaches: build the application (e.g., a JAR file), package it into a Docker image, and deploy it to a server or cloud platform (AWS, Azure, Kubernetes, etc.), often through a CI/CD pipeline that automates building, testing, and deploying on every code change.

92. How do you check whether your deployed application is working correctly? Check health-check endpoints (Spring Boot Actuator's `/actuator/health` is common), monitor application logs for

errors, test key API endpoints manually or with automated smoke tests, and monitor metrics/dashboards (CPU, memory, response times) after deployment.

93. How do you manage your code using Git/GitHub? Code is version-controlled using Git — you create feature branches for new work, commit changes with clear messages, push to GitHub, and open Pull Requests for code review before merging into the main branch. This keeps a full history of changes and allows collaboration without overwriting others' work.

---

8. Project / Behavioral / HR

94. Tell me about yourself. Give a brief, structured summary: your current role/experience, key technologies you work with, a highlight project or achievement, and why you're excited about this opportunity. Keep it under 1-2 minutes and relevant to the job you're applying for.

95. What is your current role and responsibility? Describe your job title, the team/project you're part of, and your day-to-day responsibilities (e.g., "I work as a backend developer, building and maintaining REST APIs using Spring Boot, and collaborating with the frontend team on integration").

96. What is your role in your current project? Explain specifically what part of the project you own — e.g., "I designed and developed the order management module, including the database schema and REST APIs" — be specific about your individual contribution.

97. Explain your current project and its architecture. Briefly describe the purpose of the project, then explain the architecture: what layers/services exist (e.g., frontend, backend APIs, database), how they communicate, and any key technologies (Spring Boot, PostgreSQL, Docker, etc.) used.

98. What technologies are you currently working with? List the specific languages, frameworks, tools, and databases you use daily (e.g., Java, Spring Boot, PostgreSQL, Docker, Git, REST APIs) — keep it accurate and relevant to what the interviewer is hiring for.

99. How did you use Spring Boot in your project? Explain concretely: "I used Spring Boot to build REST APIs for [feature], used Spring Data JPA to connect to PostgreSQL, and used `@Transactional` to manage database consistency for [specific operation]." Concrete, specific answers are more convincing than generic ones.

100. Why are you looking for a change? Keep it positive and forward-looking — focus on what you're looking *for* (growth, new challenges, better technology exposure) rather than complaining about your current company or manager.

101. Why should we hire you? Connect your specific skills and experience directly to what the role needs — mention relevant technical skills, past achievements, and your eagerness/fit for their team or product domain.

102. Do you have any questions for us? Always have a few ready — e.g., "What does the team's day-to-day work look like?", "What technologies/tools does the team use?", "What are the biggest challenges the team is currently facing?" Asking questions shows genuine interest.